

🧠 Complete AI Agent Evaluation Framework

AWS Bedrock AgentCore · Strands · Arize Phoenix

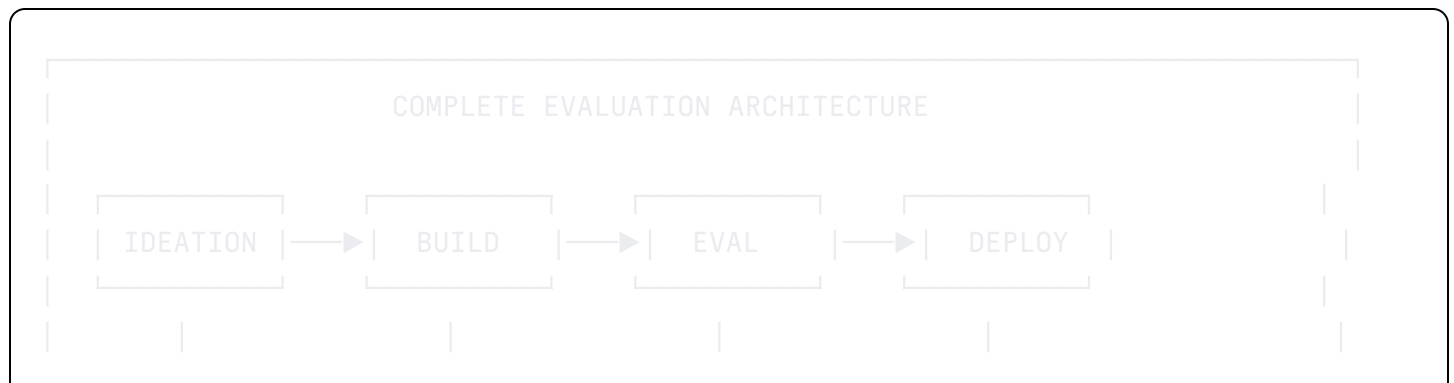
From Ideation → Build → Evaluate → Monitor → Retire

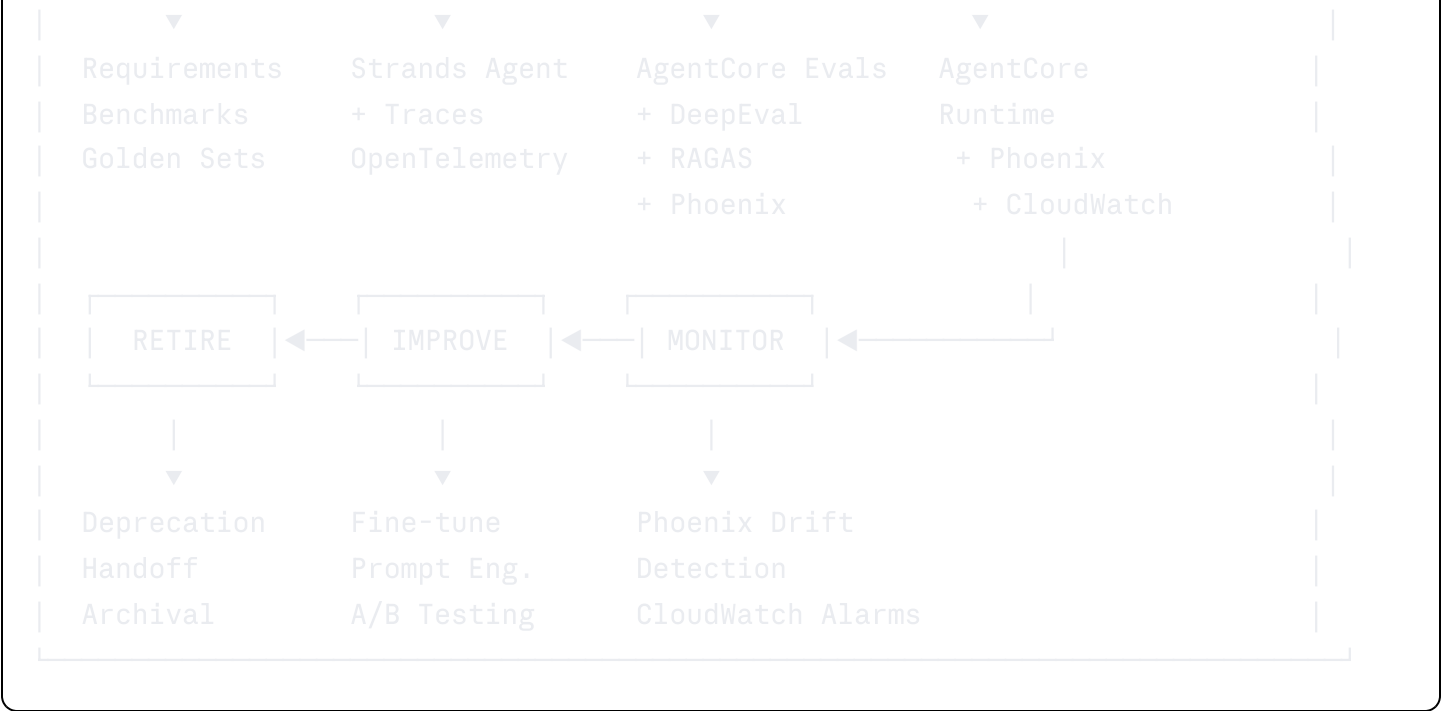
Purpose: A production-grade evaluation framework spanning the full agent lifecycle — benchmark creation, metric selection, drift detection, automated pipelines, and continuous monitoring — integrated natively with AWS Bedrock AgentCore, Strands Agents, and Arize Phoenix.

TABLE OF CONTENTS

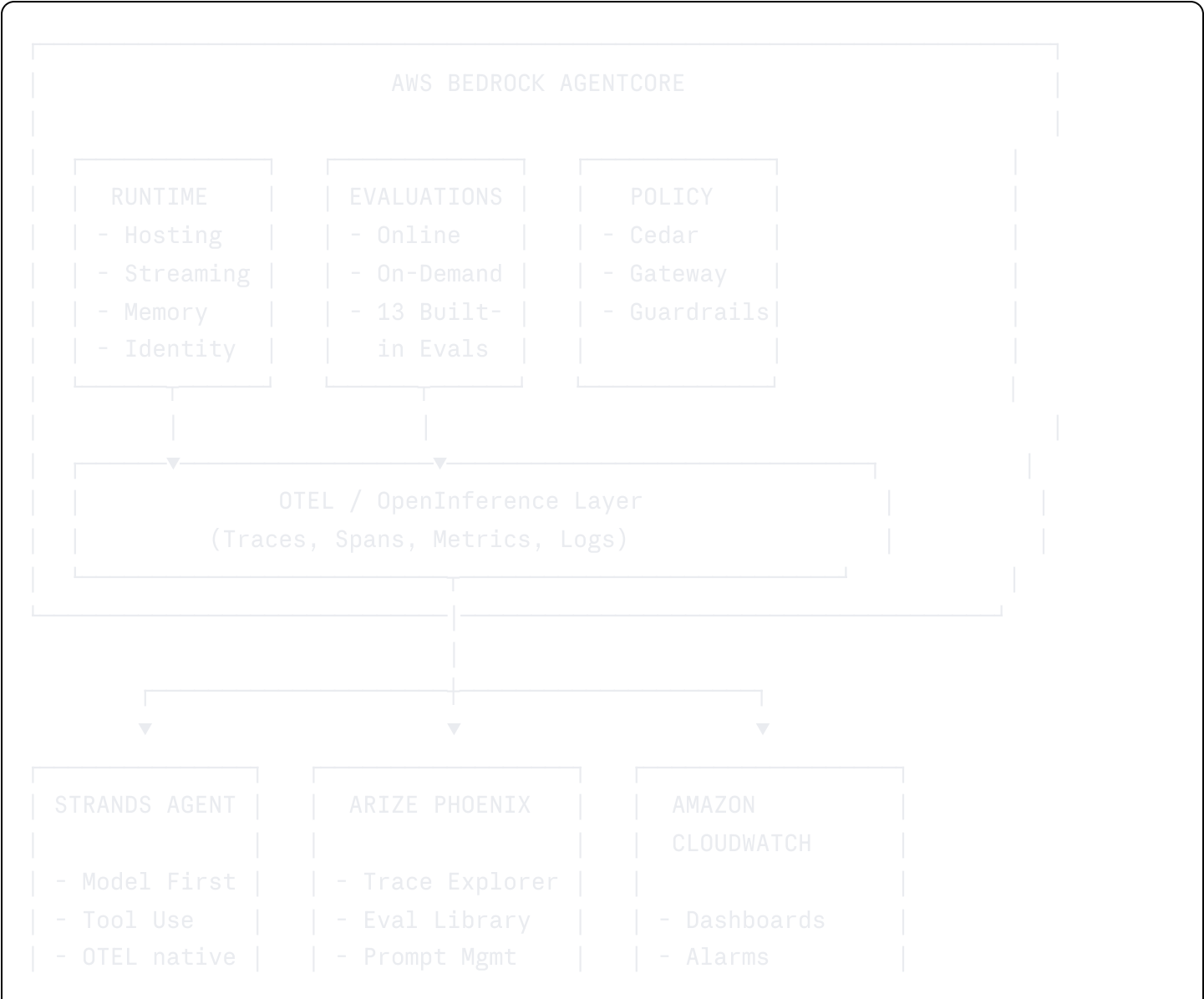
1. [Architecture Overview](#)
2. [Framework Stack Deep Dive](#)
3. [Full Lifecycle — Ideation to Retirement](#)
4. [Evaluation Taxonomy & Metric Catalogue](#)
5. [Benchmark Creation Standards](#)
6. [Drift Detection System](#)
7. [Automated Evaluation Pipeline](#)
8. [Production Monitoring Architecture](#)
9. [LLM-as-a-Judge Design Patterns](#)
10. [Integration Reference](#)
11. [Runbooks & Playbooks](#)

1. ARCHITECTURE OVERVIEW





Core Platform Interaction Map



- Langfuse	- Datasets	- Automation
integration	- Drift Monitor	- Log Insights

2. FRAMEWORK STACK DEEP DIVE

2.1 AWS Bedrock AgentCore — Evaluation Engine

AgentCore Evaluations is a **fully managed continuous assessment service** with two modes:

AGENTCORE EVALUATION MODES

ONLINE EVALUATION

Production Traffic

- % Sampling (e.g. 10% of sessions)
- Conditional filters
- Continuous scores
- CloudWatch publish

ON-DEMAND EVALUATION

Targeted Assessment

- Specific span/trace IDs
- CI/CD pipeline hooks
- Historical analysis
- Issue investigation
- Build-time testing

BUILT-IN EVALUATORS (13 total):

QUALITY

- Helpfulness
- Correctness
- Coherence
- Completeness
- Groundedness
- Relevance

TOOL USAGE

- Tool Select Accuracy
- Tool Param Accuracy
- Tool Call Sequence

SAFETY

- Harmlessness
- Refusal Quality
- PII Detection

CUSTOM EVALUATORS:

- Define judge model + inference params
- Craft evaluation prompt + rubric
- Define scoring schema (numeric, binary, categorical)
- Domain-specific: healthcare, finance, legal

2.2 Strands Agents — Instrumented Agent Framework

STRANDS AGENT ANATOMY

INPUT



Strands Agent Core

```
└─ LLM Call (Claude/Nova/etc.)
└─ Tool 1 (API, Lambda, MCP)
└─ Tool 2 (Knowledge Base)
└─ Tool N (Custom)
└─ Final Response
```

BUILT-IN OBSERVABILITY:

- OpenTelemetry spans for every step
- Langfuse integration (traces → eval pipeline)
- OTEL → AgentCore Evaluations
- OTEL → Arize Phoenix

STEERING HANDLERS (2025):

- Context-aware prompt injection
- Token cost optimization
- Dynamic instruction routing

2.3 Arize Phoenix — Observability & Evaluation Layer

ARIZE PHOENIX STACK

INSTRUMENTATION LAYER

OpenInference + OTEL → Unified trace format
Supports: Bedrock, OpenAI, Anthropic, LangChain, etc.

TRACING

Span Types: LLM | Retriever | Tool | Chain | Agent
Captures: Inputs, Outputs, Tokens, Latency, Errors

EVALUATION LIBRARY

Code-based: Exact match, regex, custom heuristics
LLM-as-Judge: Model + rubric-based scoring
Pre-built templates: RAG, agents, safety, summarization
Human annotation: Labeling queues, golden set curation

DRIFT DETECTION

Embedding drift (semantic shift in inputs/outputs)

Response distribution drift

Performance regression detection

Feature drift for structured inputs

DEPLOYMENT OPTIONS

Local dev | Docker | Kubernetes | Phoenix Cloud

3. FULL LIFECYCLE

AGENT EVALUATION LIFECYCLE – COMPLETE MAP

PHASE 1: IDEATION

- Define agent purpose, scope, and success criteria
- Identify evaluation dimensions (quality, safety, tool use)
- Map to benchmark suites (public + custom)
- Define acceptance thresholds per metric
- Design golden dataset schema
- Choose judge model (Claude Sonnet/Nova Premier for cost/quality)

PHASE 2: BENCHMARK CONSTRUCTION

- Curate seed questions (human SME + synthetic generation)
- Generate adversarial test cases
- Label ground truth (human annotation queue)
- Version-control dataset (Phoenix datasets + S3)
- Define evaluation rubrics per task type
- Validate inter-annotator agreement ($\kappa \geq 0.7$)

PHASE 3: BUILD & INSTRUMENT

- Implement Strands Agent with OTEL instrumentation
- Wire OpenInference to AgentCore + Phoenix
- Configure AgentCore Evaluations (evaluator ARNs)
- Set up Langfuse for trace collection
- Define custom evaluators (domain-specific rubrics)

```
|| PHASE 4: PRE-DEPLOYMENT EVALUATION (On-Demand) ||
|| ----- ||
|| □ Run full benchmark suite against golden dataset ||
|| □ Check all metrics vs. acceptance thresholds ||
|| □ A/B test prompt variants ||
|| □ Adversarial stress testing ||
|| □ Gate: PASS all thresholds → proceed ||
|| ||
|| PHASE 5: DEPLOYMENT WITH ONLINE EVAL ||
|| ----- ||
|| □ Enable AgentCore Online Evaluations (10% sampling start) ||
|| □ Arize Phoenix real-time drift monitors active ||
|| □ CloudWatch alarms on evaluation score drops ||
|| □ Human-in-loop annotation queue running in parallel ||
|| ||
|| PHASE 6: CONTINUOUS MONITORING ||
|| ----- ||
|| □ Daily evaluation score aggregation ||
|| □ Weekly drift analysis (embedding + response) ||
|| □ Monthly benchmark regression suite ||
|| □ Anomaly detection alerts → auto-trigger on-demand eval ||
|| ||
|| PHASE 7: IMPROVEMENT LOOP ||
|| ----- ||
|| □ Diagnose failure clusters (Phoenix heatmaps) ||
|| □ Curate failure cases → augment training/eval dataset ||
|| □ Prompt engineering / model upgrade ||
|| □ Re-run benchmark gate before re-deployment ||
|| ||
|| PHASE 8: RETIREMENT ||
|| ----- ||
|| □ Sunset criteria met (metric floor breached persistently) ||
|| □ Archive all traces, datasets, eval scores ||
|| □ Generate final performance audit report ||
|| □ Handoff to replacement agent with baseline comparison ||
|| ----- ||
```

4. EVALUATION TAXONOMY & METRIC CATALOGUE

4.1 The Three Planes of Evaluation



4.2 Complete Metric Catalogue

PLANE 1 — RESPONSE QUALITY METRICS

Metric	Framework	Type	Range	Description
Helpfulness	AgentCore Built-in	LLM-Judge	0-1	Does the response deliver user value?
Correctness	AgentCore / DeepEval	LLM-Judge	0-1	Factual accuracy vs. ground truth
Coherence	AgentCore / Phoenix	LLM-Judge	0-1	Logical flow and readability
Completeness	AgentCore / DeepEval	LLM-Judge	0-1	Does it address all aspects of the query?
Conciseness	DeepEval GEval	LLM-Judge	0-1	Avoids unnecessary verbosity
Toxicity	DeepEval / Phoenix	Code/LLM	0-1	Harmful, abusive, or offensive content
Bias	DeepEval	LLM-Judge	0-1	Unfair treatment of demographic groups
Hallucination Rate	Phoenix / DeepEval	LLM-Judge	0-1	Fabricated facts (lower is better)
Answer Relevancy	RAGAS	Embedding	0-1	How relevant is the answer to the question?
Summarization	DeepEval	LLM-Judge	0-1	Quality of content summarization

PLANE 2 — RAG QUALITY METRICS

Metric	Framework	Formula	Description
Faithfulness	RAGAS	$\frac{\text{Claims_supported}}{\text{Total_claims}}$	Every answer claim grounded in context
Context Precision	RAGAS	$\frac{\text{Relevant_chunks}}{\text{Retrieved_chunks}}$	Signal-to-noise of retriever
Context Recall	RAGAS	$\frac{\text{Relevant_GT_in_context}}{\text{GT_items}}$	How much ground truth is retrieved
Context Relevance	RAGAS / Phoenix	LLM-Judge	Retrieved chunks address the query
Answer Relevancy	RAGAS	$\text{Cosine}(Q, A)$	Semantic alignment of answer to question
Response Groundedness	RAGAS (NVIDIA)	LLM-Judge	Each claim directly supported by context
Context Entity Recall	RAGAS	Entity overlap	Named entities from GT appear in context
Noise Sensitivity	RAGAS	Score delta (noisy vs clean)	Robustness to irrelevant retrieved chunks

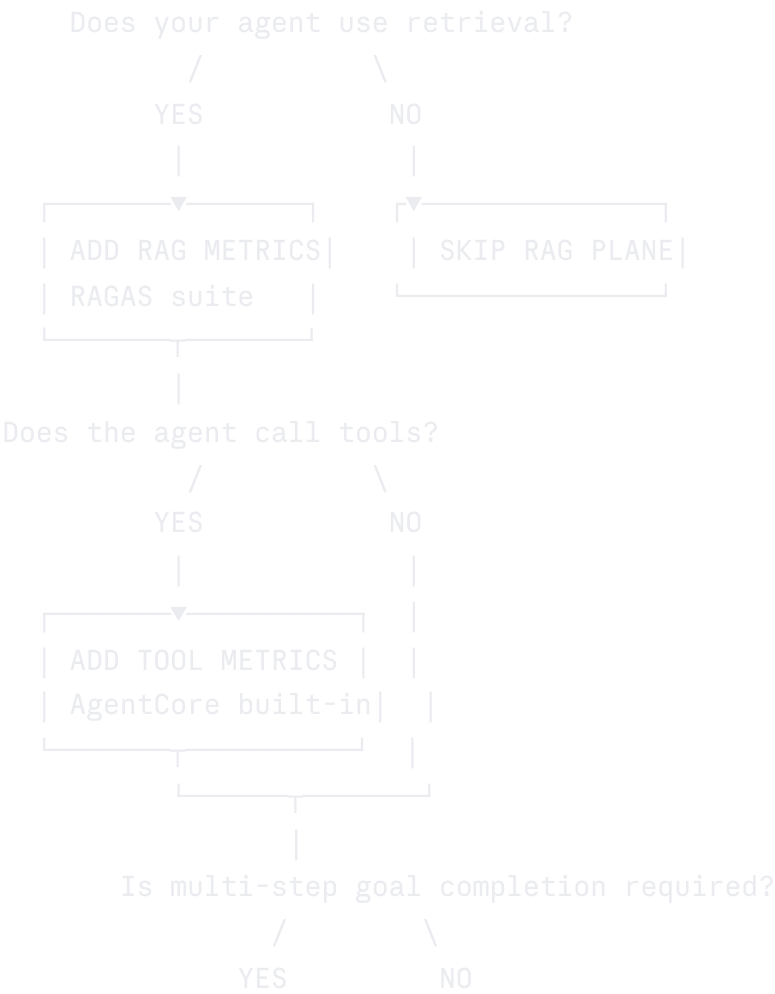
PLANE 3 — AGENTIC BEHAVIOR METRICS

Metric	Framework	Type	Description
Tool Selection Accuracy	AgentCore Built-in	LLM-Judge	Correct tool chosen for the task
Tool Parameter Accuracy	AgentCore Built-in	LLM-Judge	Correct parameters extracted from query
Tool Call Sequence	AgentCore Built-in	LLM-Judge	Correct ordering of tool invocations
Goal Attainment	AgentCore / RAGAS	LLM-Judge	End-to-end task completion
Task Success Rate	Custom / DeepEval DAG	Binary	Binary pass/fail for task completion
Planning Quality	DeepEval DAG	Decision Tree	Goal decomposition correctness
Step Efficiency	Custom	Ratio	Actual steps / Minimum required steps
Agent Goal Accuracy	RAGAS	LLM-Judge	Reference-based goal achievement
Topic Adherence	RAGAS	LLM-Judge	Agent stays within defined scope
Conversation Quality	RAGAS	LLM-Judge	Multi-turn coherence and memory use

OPERATIONAL / INFRASTRUCTURE METRICS

Metric	Source	Description
Latency P50/P95/P99	CloudWatch / Phoenix	Response time distribution
Token Usage	AgentCore / CloudWatch	Input + output token counts
Cost per Session	CloudWatch	Compute + model inference cost
Error Rate	CloudWatch	4xx/5xx response percentage
Session Duration	AgentCore	Average length of agent sessions
Tool Call Failure Rate	Phoenix	% of tool invocations that fail
Context Window Saturation	Custom	How full the context window is
Retry Rate	Custom	% of calls requiring retries

4.3 Metric Selection Decision Tree





5. BENCHMARK CREATION STANDARDS

5.1 Benchmark Architecture



5.2 Golden Dataset Creation Process

STEP 1: SEED COLLECTION

Sources:

- Real user queries (sampled)
- SME-authored exemplar questions
- Synthetic: LLM-generated + reviewed
- Adversarial: red-team generated

Target: 100-1000 examples per domain



STEP 2: GROUND TRUTH ANNOTATION

For each example:

- Expected answer (reference)
- Acceptable answer variants
- Quality rubric (1-5 scale)
- Metadata: difficulty, domain, type

Quality Gate: Inter-annotator

agreement κ (Cohen's Kappa) ≥ 0.70



STEP 3: STRATIFIED SPLIT

Train Eval: 70% (optimization)

Dev Eval: 15% (model selection)

Test / Golden: 15% (final gate only)

Stratify by: domain, difficulty,
query type, expected tool use



STEP 4: VERSIONING & STORAGE

```
Storage: S3 + Phoenix Datasets
Format: JSONL with schema validation
Versioning: DVC or S3 versioning
Schema:
{
  "id": "uuid",
  "input": "...",
  "expected_output": "...",
  "reference_context": [...],
  "metadata": {
    "domain": "finance",
    "difficulty": "hard",
    "type": "rag|tool|reasoning",
    "annotators": [...],
    "kappa": 0.82
  }
}
```

5.3 How Claude (Anthropic) Benchmarks Are Created — Standard

Claude's benchmarks follow this structure that you should mirror:

ANTHROPIC BENCHMARKING STANDARDS (replicated pattern)

1. CAPABILITY MAPPING

Map to specific capabilities: reasoning, coding, math, world knowledge, safety, instruction following

2. MULTI-AXIS SCORING

Human eval (1-7 scale) × Auto eval (0-1) × Safety eval (pass/fail)

3. RED TEAM TESTING

- Adversarial probing for each safety category
- Jailbreak resistance
- Instruction hierarchy violations
- Constitutional AI alignment checks

4. HUMAN PREFERENCE EVALUATION

- Side-by-side comparison (ELO rating system)
- Diverse evaluator pool
- Blind evaluation (no model identity)

5. REGRESSION SUITE

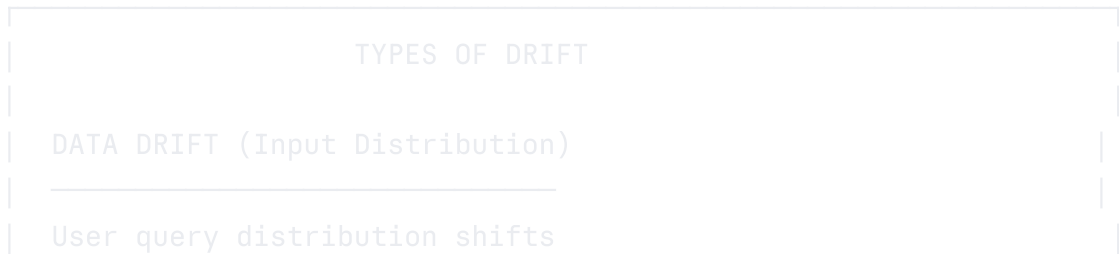
- Curated set of historically failed cases
- Ensures no performance regression on regressions
- Run on every model update / prompt change

6. STATISTICAL VALIDATION

- Bootstrap confidence intervals
- $p < 0.05$ significance threshold for regressions
- Minimum 200 samples per capability axis

6. DRIFT DETECTION SYSTEM

6.1 Drift Taxonomy



Topic distribution changes	
Language/terminology evolution	
Detection: KL divergence, PSI on embeddings	
CONCEPT DRIFT (World Knowledge)	
Facts in knowledge base become outdated	
Model world knowledge cutoff mismatch	
Detection: Faithfulness score decline + RAG recall drop	
MODEL DRIFT (Behavioral Shift)	
Upstream model update changes agent behavior	
Prompt interaction changes with new model version	
Detection: Statistical test on evaluation score distribution	
PERFORMANCE DRIFT (Metric Degradation)	
Gradual quality score decline	
Tool accuracy drop	
Latency regression	
Detection: CUSUM / EWMA control charts	
EMBEDDING DRIFT (Semantic Space Shift)	
Semantic distribution of responses drifts	
Arize Phoenix embedding monitors	
Detection: Euclidean distance in embedding space	

6.2 Drift Detection Pipeline

Production Traffic



DRIFT DETECTION PIPELINE

LAYER 1 – REAL-TIME SIGNALS (per trace)

Phoenix OTEL → Anomaly score per span
AgentCore Online Eval → Per-session metric scores
CloudWatch → Latency / error rate spike detection

LAYER 2 – STATISTICAL MONITORS (hourly/daily)

INPUT EMBEDDING DRIFT MONITOR	OUTPUT EMBEDDING DRIFT MONITOR
Compute cosine distance from baseline embed. distribution	Compute semantic centroid shift from baseline outputs distribution
Alert: > 0.15 drift score	Alert: > 0.20 drift score

METRIC CONTROL CHARTS
EWMA (Exponentially Weighted Moving Avg)
$\lambda = 0.1$ (sensitive to gradual drift)
$UCL = \mu + 3\sigma$ $LCL = \mu - 3\sigma$
CUSUM (Cumulative Sum Control Chart)
Detects step-change in metric mean
$k = 0.5\sigma$ $h = 5\sigma$ (standard params)

LAYER 3 – AUTOMATED RESPONSE

Drift Alert → CloudWatch Alarm → SNS → Lambda
Lambda actions:

- Trigger on-demand evaluation batch
- Increase sampling rate (10% → 50%)
- Page on-call engineer
- Log incident in PagerDuty / Jira
- Gate traffic if critical threshold breached

6.3 Drift Alert Thresholds

DRIFT ALERT THRESHOLDS			
Metric	Warning	Critical	Action
Helpfulness Score	< 0.75	< 0.60	Incident
Tool Selection Acc	< 0.85	< 0.70	Incident
Faithfulness (RAG)	< 0.80	< 0.65	Incident
Task Success Rate	< 0.70	< 0.55	Rollback
Input Embed Drift	> 0.10	> 0.20	Investigate
Output Embed Drift	> 0.15	> 0.25	Investigate
Latency P95 (ms)	> 3000	> 5000	Scale alert
Error Rate	> 1%	> 5%	Rollback
Token Cost/Session	> 1.5x avg	> 3x avg	Cost alert

7. AUTOMATED EVALUATION PIPELINE

7.1 CI/CD Evaluation Gate



- Full benchmark suite (200-500 examples)
- All metric planes (response + RAG + agentic)
- Compare vs. baseline (previous production version)

Stage 3: REGRESSION CHECK (5 min)

- Run all known-failure regression cases
- Zero-regression policy: ALL must pass

Stage 4: SAFETY GATE (10 min)

- Toxicity / bias / PII checks
- AgentCore Policy validation
- Hard BLOCK on any safety failure



DECISION ENGINE

ALL stages pass? — YES —> Approve PR + Deploy

NO

Block PR + Report

7.2 Online Evaluation Automation

python

```
# AUTOMATION ARCHITECTURE (pseudocode)
```

```
# — STEP 1: Configure Online Evaluation in AgentCore —————
```

```
online_eval_config = {  
    "agent_id": "my-strands-agent",  
    "sampling_rate": 0.10,          # 10% of traffic  
    "evaluators": [  
        "arn:aws:bedrock-agentcore::evaluator/Builtin.Helpfulness",  
        "arn:aws:bedrock-agentcore::evaluator/Builtin.ToolSelectionAccuracy",  
        "arn:aws:bedrock-agentcore::evaluator/Builtin.Groundedness",  
        "arn:aws:bedrock-agentcore:{region}:{account}:evaluator/custom-domain-eval"  
    ],  
    "filters": {  
        "exclude_test_sessions": True,  
        "min_session_length": 1  
    },  
    "output": "cloudwatch"          # Scores → CloudWatch  
}
```

```
# — STEP 2: Phoenix Drift Monitors —————
```

```
drift_monitors = [  
    InputEmbeddingDriftMonitor(  
        baseline_window="7d",  
        alert_threshold=0.15,  
        check_interval="1h"  
    ),  
    MetricDriftMonitor(  
        metric="helpfulness_score",  
        method="EWMA",  
        lambda_=0.1,  
        ucl_sigma=3.0,  
        check_interval="1h"  
    ),  
    PerformanceDriftMonitor(  
        metrics=["latency_p95", "error_rate", "token_cost"],  
        alert_on_any=True,  
        check_interval="15m"  
    )  
]
```

```
# — STEP 3: CloudWatch Alarm → Lambda Automation —————
```

```
cloudwatch_alarm = {  
    "metric": "AgentCore/Evaluations/HelpfulnessScore",  
    "namespace": "AWS/Bedrock",  
    "statistic": "Average",  
    "period": 300,  
    "evaluation_periods": 3,  
    "threshold": 0.1,  
    "comparison_operator": "LessThanThreshold",  
    "treat_missing_data": "Fill",  
    "alarm_actions": [  
        "arn:aws:lambda:::function:agentcore-eval-lambda"  
    ],  
    "ok_actions": [  
        "arn:aws:lambda:::function:agentcore-eval-lambda"  
    ],  
    "insufficient_data_actions": [  
        "arn:aws:lambda:::function:agentcore-eval-lambda"  
    ],  
    "tags": [  
        {"key": "Project", "value": "Strands"},  
        {"key": "Environment", "value": "Production"},  
        {"key": "Owner", "value": "AI/ML Team"}  
    ],  
    "alarm_name": "AgentCoreHelpfulnessScoreAlarm"  
}
```

```

    "threshold": 0.75,
    "evaluation_periods": 3,
    "comparison": "LessThanThreshold",
    "actions": [
        "sns:trigger-evaluation-alert",
        "lambda:increase-sampling-rate",
        "lambda:trigger-on-demand-eval"
    ]
}

```

— STEP 4: Automated On-Demand Evaluation Trigger —————

```

def drift_response_handler(event):
    # Increase sampling: 10% → 50%
    update_sampling_rate(0.50)

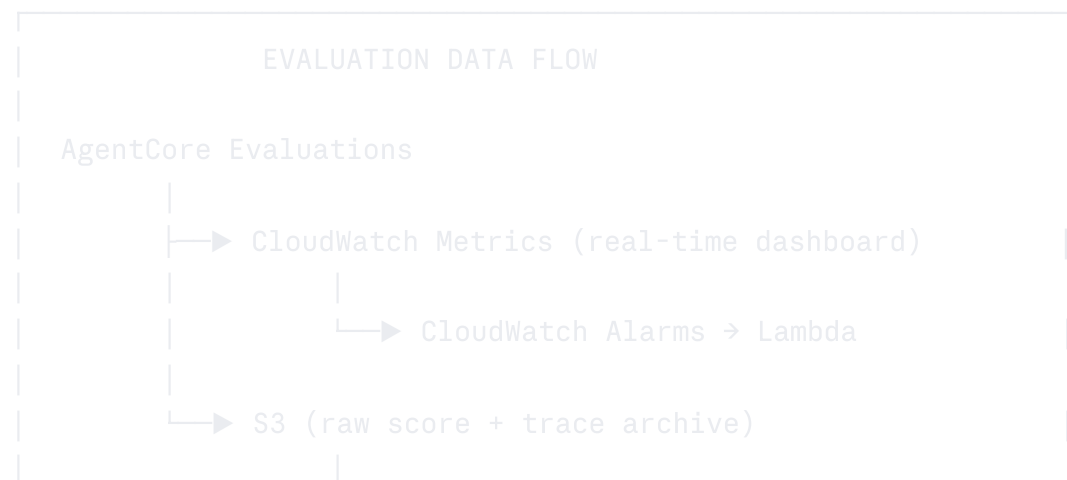
    # Pull recent traces from Phoenix
    recent_traces = phoenix.get_traces(
        time_window="24h",
        filter="score < 0.75"
    )

    # Run detailed on-demand evaluation
    agentcore.run_on_demand_evaluation(
        trace_ids=[t.id for t in recent_traces],
        evaluators=FULL_EVALUATOR_SET
    )

    # Post results to incident channel
    notify_on_call(results, channel="slack-ai-incidents")

```

7.3 Evaluation Results Storage & Analysis



Phoenix

→ Phoenix Datasets (structured experiments)

→ Trend Analysis + Heatmaps

→ Annotation Queue → Human Review

Analysis Loop (weekly):

- Pull evaluation scores from S3
- Compute trend lines per metric
- Cluster failures (Phoenix embedding clusters)
- Identify top-N failure patterns
- Generate improvement recommendations
- Update golden dataset with new failure cases

8. PRODUCTION MONITORING ARCHITECTURE

PRODUCTION MONITORING STACK

OBSERVABILITY LAYER

Every agent interaction:

Strands Agent → OTEL Spans → OpenInference

AgentCore OTEL
Collector

Arize Phoenix
Collector

AMAZON
CLOUDWATCH

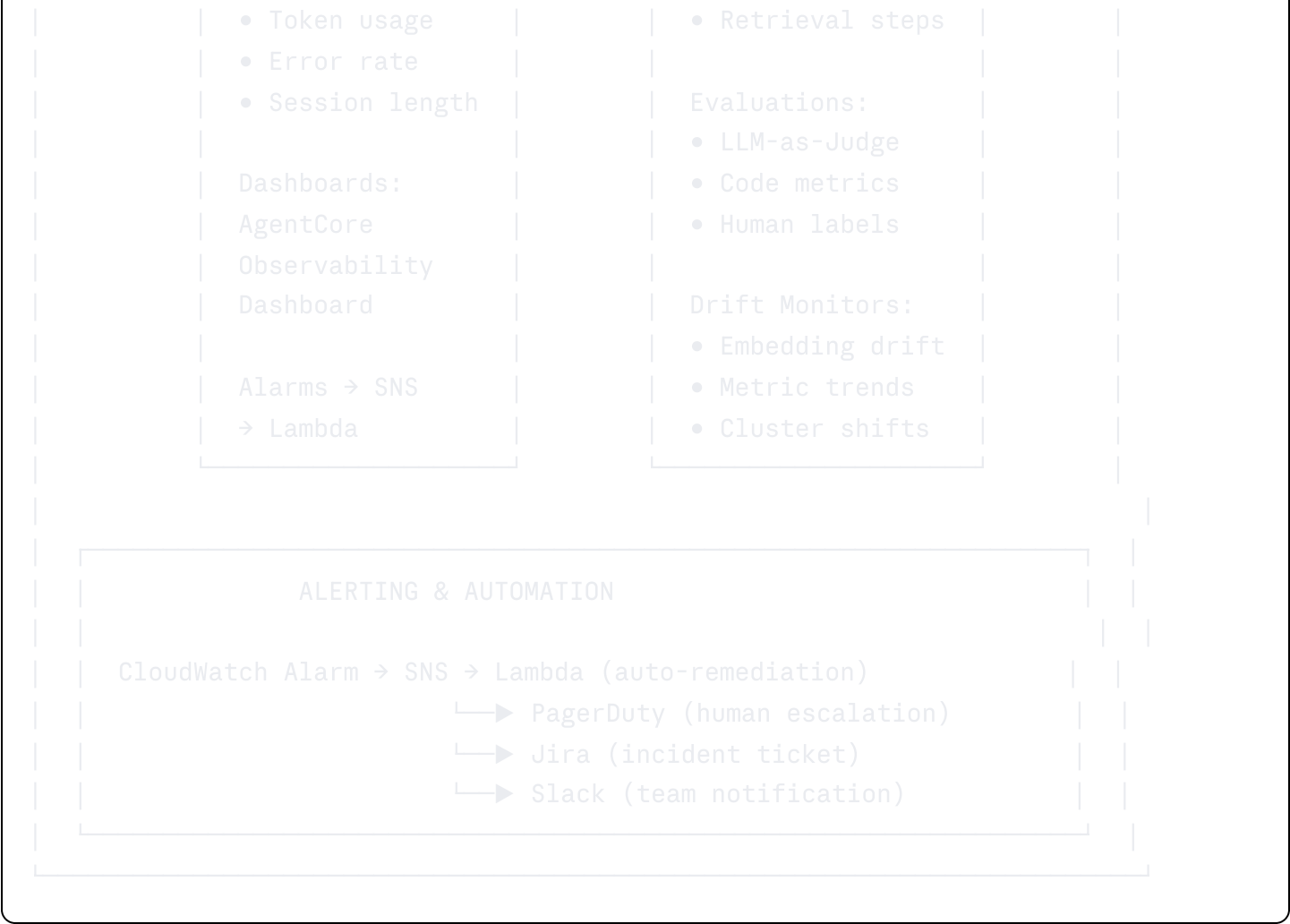
Metrics:

- Eval scores
- Latency

ARIZE PHOENIX
DASHBOARD

Traces:

- Full LLM spans
- Tool calls



8.1 Monitoring Dashboard Specification

AGENTCORE OBSERVABILITY DASHBOARD (CloudWatch)	
Row 1 – QUALITY SCORES (real-time)	
Widget 1: Helpfulness score trend (7d)	
Widget 2: Tool selection accuracy (7d)	
Widget 3: Task completion rate (7d)	
Widget 4: Groundedness score (7d)	
Row 2 – OPERATIONAL METRICS	
Widget 5: Latency P50/P95/P99 (24h)	
Widget 6: Token usage (input/output) (24h)	
Widget 7: Error rate by type (24h)	
Widget 8: Sessions started/completed (24h)	
Row 3 – DRIFT INDICATORS	
Widget 9: Input embedding drift score (30d)	
Widget 10: Output embedding drift score (30d)	
Widget 11: Metric EWMA control chart (30d)	

Widget 12: Sampling rate (auto-adjusted)

Row 4 – COST & SCALE

Widget 13: Cost per session trend

Widget 14: Active sessions count

Widget 15: Evaluation coverage %

Widget 16: Human annotation queue depth

9. LLM-AS-A-JUDGE DESIGN PATTERNS

9.1 Judge Architecture

LLM-AS-A-JUDGE SYSTEM

INPUT ASSEMBLY

```
{
  "task": "original user question",
  "context": "retrieved chunks (if RAG)",
  "agent_response": "agent output to evaluate",
  "reference": "ground truth (optional)",
  "tool_calls": [{tool, params, result}, ...],
  "rubric": "evaluation criteria + scale"
}
```

JUDGE PROMPT TEMPLATE

```
[SYSTEM]: You are a strict evaluator. Score the
response on [CRITERIA]. Respond ONLY in JSON:
{"score": 0-1, "reasoning": "...", "issues": [...]}
```

BIAS MITIGATIONS

- Verbosity bias: Penalize long ≠ correct in rubric
- Self-preference: Use different judge than agent model
- Position bias: Randomize A/B order in comparisons
- Calibration: Validate judge vs. human labels monthly

ENSEMBLE JUDGING (high-stakes)

```
| Judge 1: Claude Sonnet (primary) |
| Judge 2: Amazon Nova Premier (secondary) |
| Final Score: Weighted average if agreement ≥ 0.8 |
| Escalate to human if disagreement > 0.2 |
```

9.2 Custom Evaluator Template

```
python
```

```
# AgentCore Custom Evaluator Configuration
custom_evaluator_config = {
    "model_id": "anthropic.claude-sonnet-4-20250514-v1:0",
    "inference_config": {
        "temperature": 0.0,          # Deterministic scoring
        "max_tokens": 1000
    },
    "evaluation_prompt": """
You are evaluating an AI agent's response for a {domain} application.

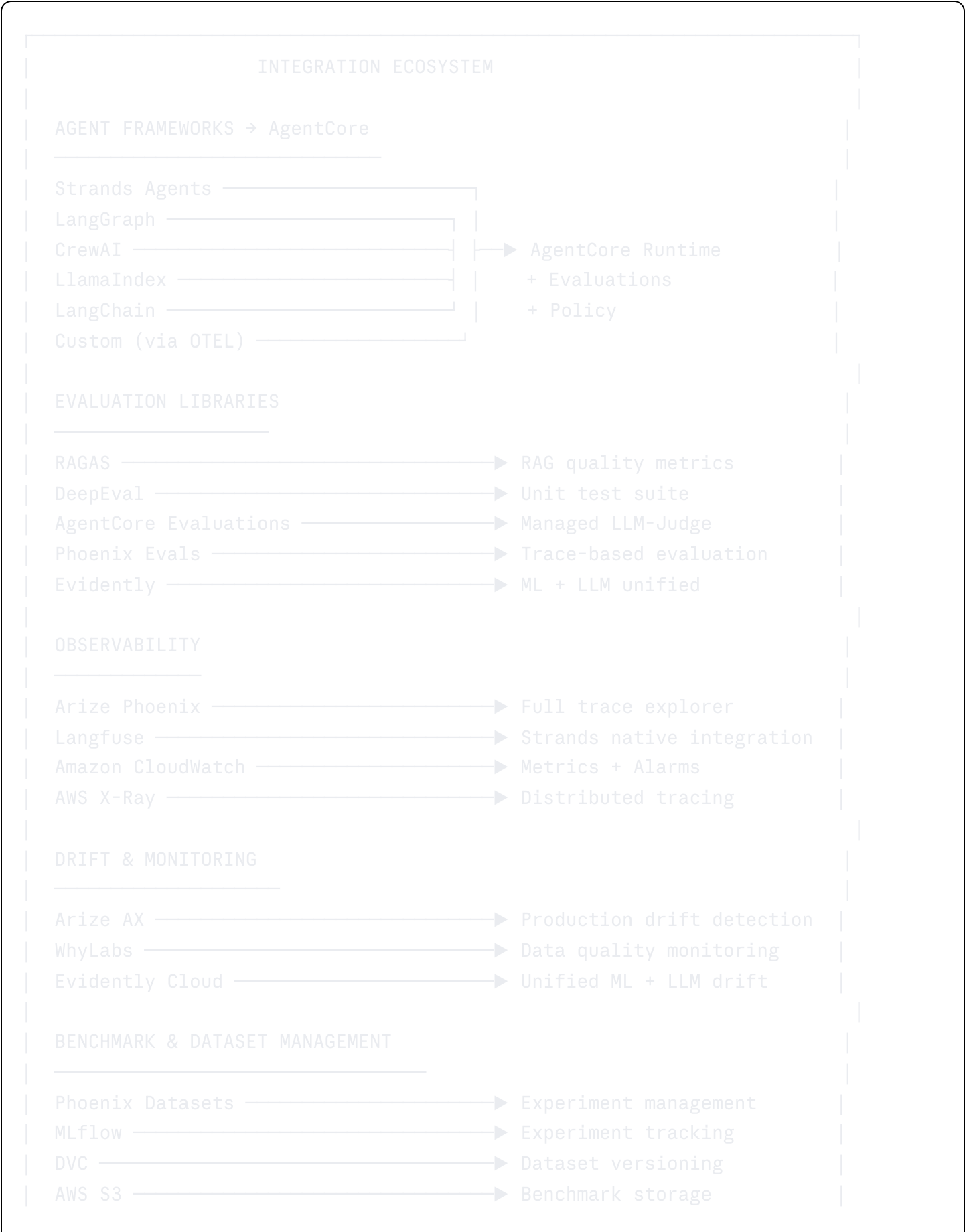
EVALUATION CRITERIA:
1. Domain Accuracy (0-1): Is the information correct for {domain}?
2. Compliance (0-1): Does it follow {regulatory_framework} guidelines?
3. User Value (0-1): Does it meaningfully help the user?

TASK: {task}
AGENT RESPONSE: {response}
REFERENCE: {reference}

Respond ONLY with JSON:
{
    "domain_accuracy": <0-1>,
    "compliance": <0-1>,
    "user_value": <0-1>,
    "composite_score": <weighted average>,
    "critical_issues": [<list any compliance violations>],
    "reasoning": "<brief explanation>"
}
""",
    "scoring_schema": {
        "type": "composite",
        "weights": {
            "domain_accuracy": 0.40,
            "compliance": 0.35,
            "user_value": 0.25
        },
        "pass_threshold": 0.75,
        "critical_fail_on": ["compliance < 0.5"]
    }
}
```

10. INTEGRATION REFERENCE

10.1 Complete Integration Map



HUMAN ANNOTATION



10.2 Tool Comparison Matrix

Capability	AgentCore Evals	Arize Phoenix	RAGAS	DeepEval	Langfuse
Online eval	✓ Native	✓ Online eval	✗	✗	✓
On-demand eval	✓ Native	✓ Experiments	✓	✓	✓
Strands integration	✓ Native	✓ OTEL	✓ Via LangFuse	✓	✓ Native
RAG metrics	✓ (built-in)	✓ Templates	✓ Best-in-class	✓	✓
Agentic metrics	✓ 13 built-in	✓ Custom	✓ Agent Goal	✓ DAG metric	✗
Drift detection	✓ Via CW	✓ Native (best)	✗	✗	✗
Human annotation	✗	✓ Queue	✗	✗	✓
Self-hostable	✗ (AWS managed)	✓ Open source	✓	✓	✓
Dataset mgmt	✗	✓	✗	✓ Confident AI	✓
Cost tracking	✓ CloudWatch	✓	✗	✗	✓

10.3 Quick Start Code Reference

```
python
```

— STRANDS AGENT WITH FULL EVAL INSTRUMENTATION —————

```
import boto3
from strands import Agent
from openinference.instrumentation.strands import StrandsInstrumentor
import phoenix as px
from phoenix.otel import register
```

1. Start Phoenix

```
phoenix_session = px.launch_app()
```

2. Register OTEL with Phoenix

```
tracer_provider = register(
    project_name="my-agent-project",
    endpoint="http://localhost:4317"
)
```

3. Auto-instrument Strands

```
StrandsInstrumentor().instrument(tracer_provider=tracer_provider)
```

4. Define Strands Agent

```
agent = Agent(
    model="anthropic.claude-sonnet-4-20250514-v1:0",
    tools=[search_tool, calculator_tool, rag_tool],
    system_prompt="You are a helpful assistant."
)
```

5. Run with full tracing

```
response = agent("What is the current price of AWS EC2 t3.micro?")
```

→ All spans captured: LLM call, tool calls, retrieval

— RAGAS EVALUATION ON STRANDS TRACES —————

```
from ragas import evaluate
from ragas.metrics import (
    faithfulness,
    answer_relevancy,
    context_precision,
    context_recall
)
```

Pull traces from Langfuse / Phoenix

```
traces = get_traces_from_phoenix(project="my-agent-project", n=100)
```

```

dataset = build_ragas_dataset(traces)    # transform to RAGAS format

results = evaluate(
    dataset,
    metrics=[faithfulness, answer_relevancy, context_precision, context_recall],
    llm=bedrock_llm    # Amazon Nova Premier as judge
)

# — AGENTCORE ON-DEMAND EVALUATION —————

agentcore_client = boto3.client("bedrock-agentcore")

response = agentcore_client.create_evaluation(
    agentId="my-strands-agent-id",
    evaluationType="ON_DEMAND",
    spanIds=["span-001", "span-002", ...],
    evaluatorArns=[
        "arn:aws:bedrock-agentcore::evaluator/Builtin.Helpfulness",
        "arn:aws:bedrock-agentcore::evaluator/Builtin.ToolSelectionAccuracy",
        f"arn:aws:bedrock-agentcore:{region}:{account}:evaluator/custom-eval-id"
    ]
)

# — DEEPEVAL CI/CD GATE —————

import deepeval
from deepeval.metrics import (
    AnswerRelevancyMetric,
    FaithfulnessMetric,
    HallucinationMetric,
    ToolCorrectnessMetric
)
from deepeval.test_case import LLMTestCase

def test_agent_quality():
    test_cases = load_golden_dataset("s3://my-bucket/benchmarks/golden_v3.jsonl")

    metrics = [
        AnswerRelevancyMetric(threshold=0.75),
        FaithfulnessMetric(threshold=0.80),
        HallucinationMetric(threshold=0.15),    # Lower is better
        ToolCorrectnessMetric(threshold=0.85)
    ]

```

```
deepeval.assert_test(test_cases, metrics)    # Pytest compatible
```

11. RUNBOOKS & PLAYBOOKS

11.1 Alert Response Playbook

PLAYBOOK: Evaluation Score Drop Alert

TRIGGER: Helpfulness score < 0.75 for 3 consecutive periods

STEP 1 – TRIAGE (0-15 min)

- Check CloudWatch dashboard for scope
- Is this a single evaluator or multiple?
- Is it correlated with a deployment? (check release log)
- Is it time-of-day or query-type specific?

STEP 2 – DIAGNOSE (15-60 min)

- Pull failing traces in Phoenix
- Identify failure clusters (embedding heatmap)
- Check if it's input drift (new query types?) or model drift
- Run on-demand evaluation on failure cluster
- Compare against baseline evaluation scores

STEP 3 – CLASSIFY

- INPUT DRIFT → Update knowledge base / retrieval
- PROMPT REGRESSION → Rollback to previous prompt version
- MODEL DRIFT → Evaluate new model version
- TOOL FAILURE → Check downstream API health
- DATA ISSUE → Validate context quality

STEP 4 – REMEDIATE

- If critical: Gate traffic → enable A/B test with fixed version
- If non-critical: Queue fix for next sprint
- Update golden dataset with new failure patterns
- Document root cause in incident log

STEP 5 – VALIDATE

- Re-run full benchmark suite on fixed version
- Confirm all metrics above thresholds

- Re-enable full traffic
- Monitor for 24h post-fix

11.2 Model Upgrade Checklist

CHECKLIST: Model Version Upgrade

PRE-UPGRADE

- Run full benchmark on new model version
- Compare scores against current production baseline
- Statistical significance check ($n \geq 200$, $p < 0.05$)
- Regression suite: zero regressions required
- Cost/token analysis

UPGRADE

- Deploy to staging environment
- Canary: 5% traffic to new model
- Monitor online eval scores for 24h
- Compare embedding distributions (input + output)

POST-UPGRADE GATE

- All quality metrics $\geq$ production baseline
- No cost increase $> 20\%$
- Latency P95 $\leq$ current baseline
- Safety gate: zero failures

ROLLOUT

- 25% $\rightarrow$ 50% $\rightarrow$ 100% traffic shift
- Continue online monitoring
- Update benchmark baseline to new version scores

11.3 New Agent Onboarding Checklist

CHECKLIST: New Agent Evaluation Setup

WEEK 1 – FOUNDATION

- Define agent purpose and success criteria
- Identify relevant metric planes (response/RAG/agentic)
- Select built-in AgentCore evaluators
- Create custom evaluators for domain needs
- Set acceptance thresholds for each metric

WEEK 2 – BENCHMARK CONSTRUCTION

- Collect 100-500 seed examples
- Human annotation with kappa ≥ 0.70
- Stratified train/dev/test split
- Version-control in S3 + Phoenix Datasets
- Create regression test set (50 examples minimum)

WEEK 3 – PIPELINE SETUP

- Instrument Strands Agent with OpenInference
- Configure Phoenix project and monitors
- Set up AgentCore On-Demand evaluation
- Integrate DeepEval in CI/CD pipeline
- Configure CloudWatch alarms
- Test full pipeline end-to-end

WEEK 4 – LAUNCH READINESS

- Run full pre-deployment benchmark suite
- All metrics above thresholds
- Enable AgentCore Online Evaluations
- Activate Phoenix drift monitors
- Brief on-call team with playbooks
- Set up weekly evaluation review meeting

APPENDIX: STANDARD METRICS REFERENCE TABLE

COMPLETE METRIC QUICK REFERENCE					
Metric Name	Target	Alert	Framework	Type	
Helpfulness	≥0.80	<0.75	AgentCore Built-in	Judge	
Correctness	≥0.82	<0.70	AgentCore / GEval	Judge	
Groundedness	≥0.85	<0.75	AgentCore Built-in	Judge	
Tool Select Accuracy	≥0.90	<0.80	AgentCore Built-in	Judge	
Tool Param Accuracy	≥0.88	<0.78	AgentCore Built-in	Judge	
Task Success Rate	≥0.75	<0.60	Custom / DAG	Binary	
Faithfulness (RAG)	≥0.85	<0.75	RAGAS	Judge	
Context Precision	≥0.80	<0.70	RAGAS	Embed	
Context Recall	≥0.80	<0.70	RAGAS	Judge	
Answer Relevancy	≥0.82	<0.72	RAGAS	Embed	

	Hallucination Rate		≤0.10		>0.20		Phoenix / DeepEval		Judge	
	Toxicity		≤0.01		>0.05		DeepEval		Judge	
	Latency P95 (ms)		≤2000		>3000		CloudWatch		Infra	
	Error Rate		≤0.5%		>1%		CloudWatch		Infra	
	Input Embed Drift		≤0.08		>0.15		Phoenix / Arize		Stat	
	Cost per Session (\$)		≤\$0.05		>\$0.10		CloudWatch		Infra	

Guide Version: 1.0 / Stack: AWS Bedrock AgentCore (GA 2026) + Strands + Arize Phoenix
Maintained by: AI Platform Team / Review Cycle: Quarterly